

Weather Data Analyzer

Project Report

Course: Introduction to Problem Solving

VITyarthi - Build Your Own Project

Submitted by: Bhagyashree Sharma

Registration No.: 26BAI10071

Date: 30/09/2026

1. Introduction

Weather stations and personal weather logs collect daily temperature readings, but raw lists of numbers are hard to interpret directly. Answering simple questions -- 'What was the hottest day?', 'How many days were unusually cold?', 'Is there a duplicate sensor entry?' -- requires systematic array-processing techniques. The Weather Data Analyzer applies exactly the array, list, and tuple algorithms covered in the Introduction to Problem Solving syllabus to turn raw readings into clear answers.

This project deliberately uses only Python lists, tuples, and manual algorithms -- no classes, no GUI framework, no external libraries -- so every line of code maps directly back to a syllabus concept: Array Order Reversal, Array Counting, Finding the Maximum/Minimum, Removal of Duplicates from an Ordered Array, Partitioning an Array, Finding the Kth Smallest Element, and the full set of List and Tuple operations.

2. Problem Statement

There is no lightweight, dependency-free tool for analyzing a small set of daily temperature readings using fundamental array algorithms. This project solves that by storing each day's reading as an immutable tuple in a list, then applying classic, hand-implemented algorithms -- rather than relying solely on built-in shortcuts -- to find extremes, detect duplicate entries, categorize and partition days, and identify the Kth ranked day by temperature.

3. Functional Requirements

The application supports the following functional modules:

- Reading Management -- add, view, and remove daily temperature readings, stored as (day, temperature) tuples in a list.
- Array Algorithms -- reverse reading order, count days by category, find hottest/coldest day, remove duplicate day entries, partition days around a threshold, and find the Kth coldest/hottest day.
- Tuple Operations -- pack/unpack readings, concatenate batches of data, slice by date range, check membership, and compare readings.
- Reporting -- generate a text summary report with an ASCII bar-chart visualization, optionally saved to a file.
- Persistence -- save and reload readings from a plain text file between sessions.

4. Non-Functional Requirements

Requirement	How it is addressed
Usability	Simple numbered menu with clear prompts; no code editing required to use the app.
Reliability	All inputs (day numbers, temperatures, K values) pass through centralized validators before use.
Error Handling	Invalid input, out-of-range K values, or missing days raise a <code>ValidationError</code> shown as a clear message, never a crash.
Maintainability	Logic is split into 7 single-responsibility modules (menu, data management, array algorithms, tuple operations, reporting, storage, validation).
Performance	All algorithms run in $O(n)$ or $O(n \log n)$ (for sorting-based operations), instant even for a full year of daily readings.
Resource Efficiency	Readings are stored as a lightweight plain-text file rather than a database.

5. System Architecture

The application follows a layered, purely functional architecture (no classes) separating the menu/CLI, validation, data management, array algorithms, tuple operations, reporting, and persistence, so each concern can be tested and modified independently.

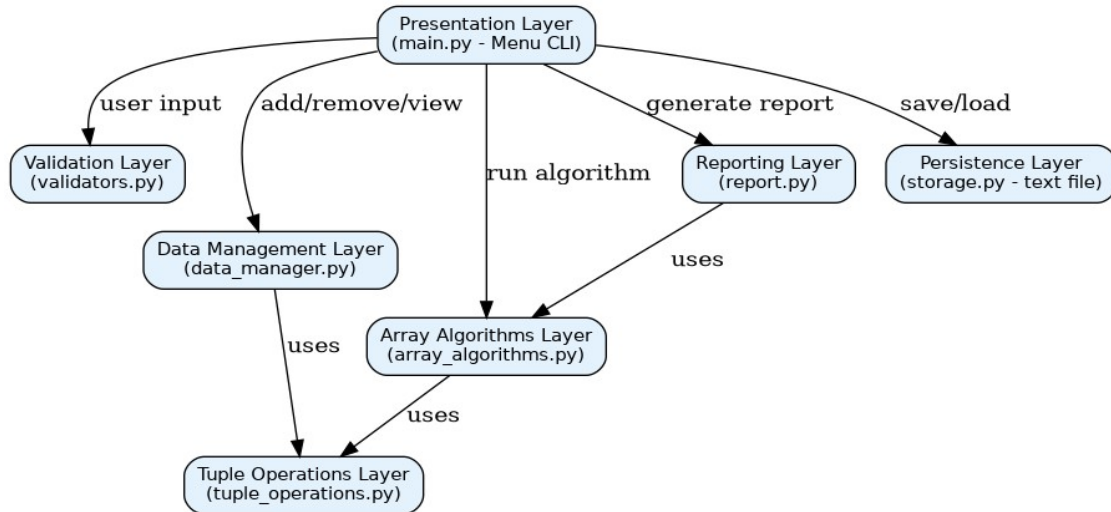


Figure 1: System Architecture Diagram

6. Design Diagrams

6.1 Process / Workflow Diagram

Shows the menu-driven flow, including the input validation checkpoint before any algorithm runs.

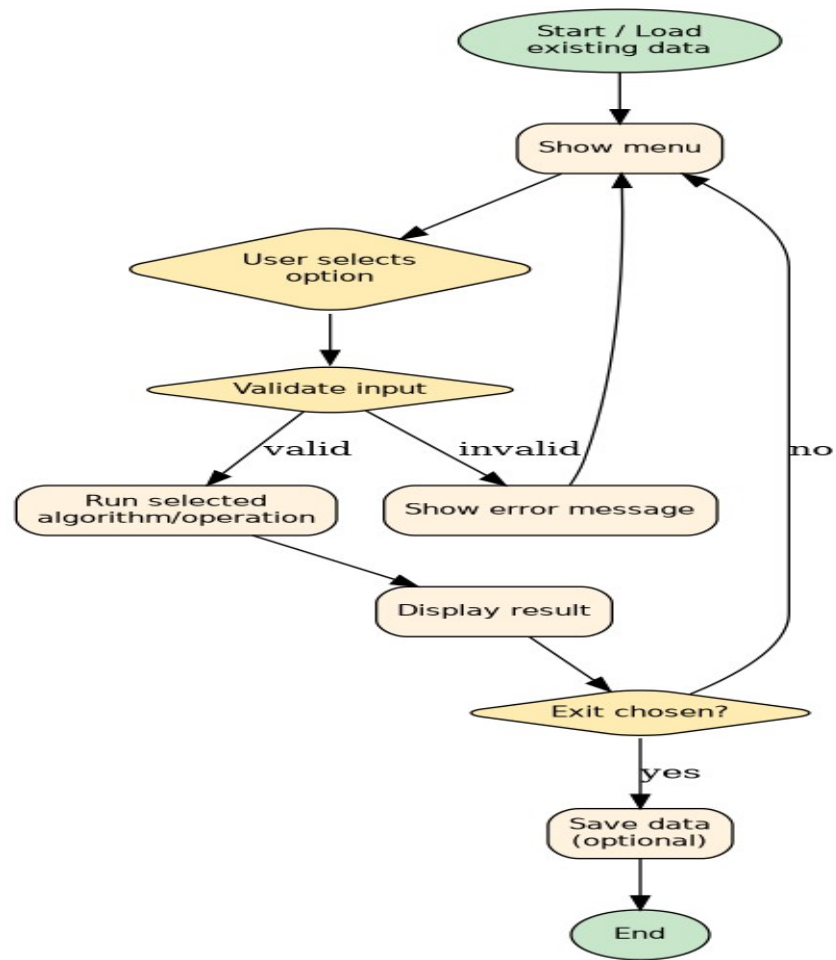


Figure 2: Process Workflow Diagram

6.2 Use Case Diagram

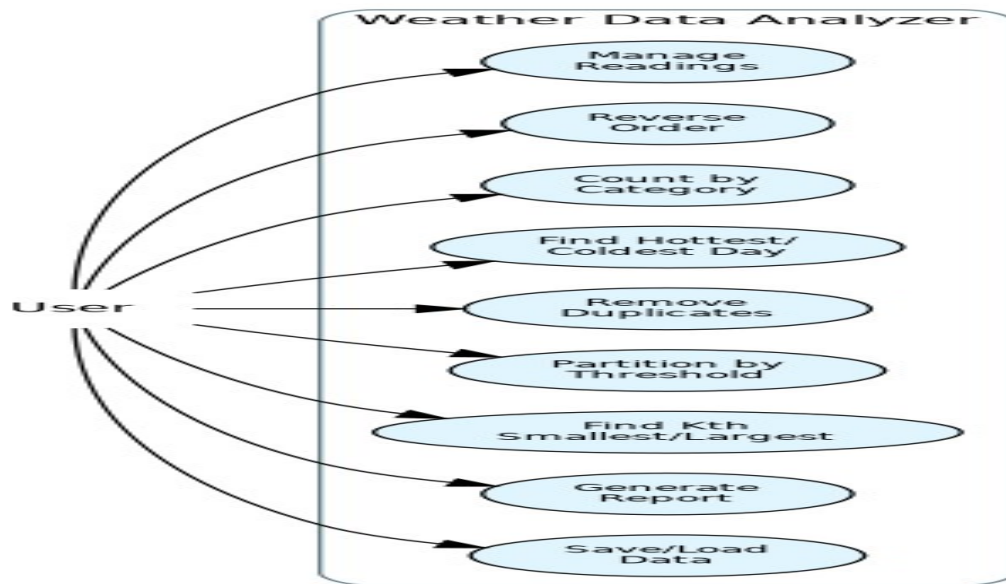


Figure 3: Use Case Diagram

6.3 Component Diagram

Since this project is implemented without classes (matching the syllabus scope), the design is shown as a component/module diagram rather than a class diagram, with each module's public functions listed.

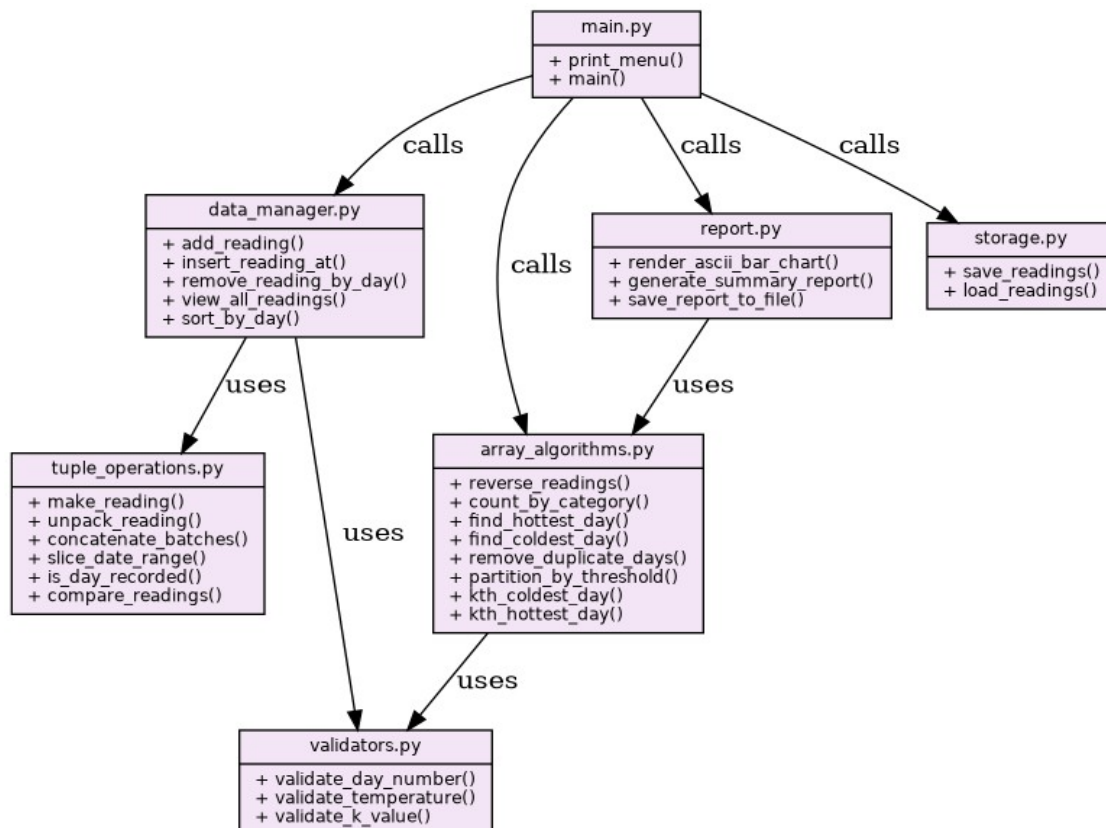


Figure 4: Component Diagram

6.4 Sequence Diagram

Illustrates the interaction for the 'Find Kth coldest day' operation (numbered left to right).

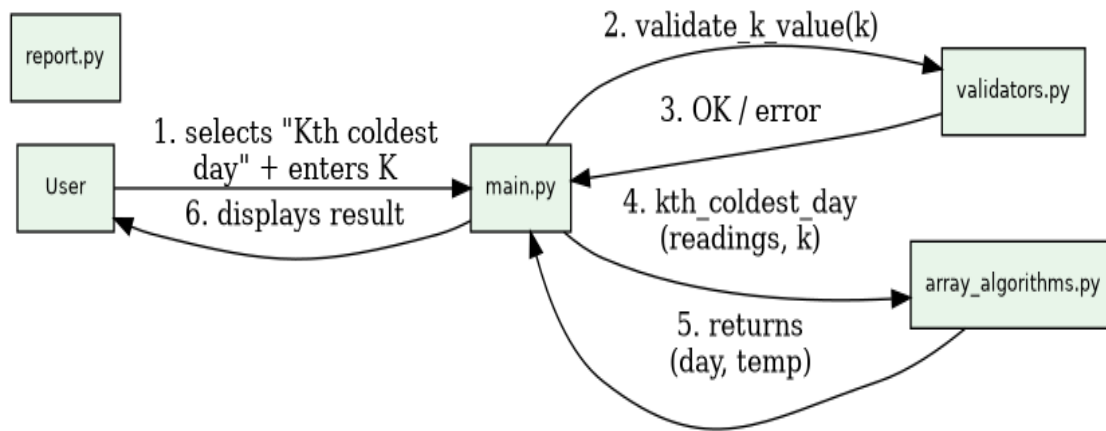


Figure 5: Sequence Diagram -- Kth Coldest Day Flow

6.5 Data Storage Design

Readings are stored in a plain CSV-style text file (one 'day,temperature' pair per line) rather than a database, since the syllabus scope covers lists/tuples/arrays rather than database design. Each line maps directly back onto a (day, temperature) tuple when loaded.

Field	Type	Description
day	int	Day number (1-365) the reading was taken
temperature	float	Recorded temperature in degrees Celsius

7. Design Decisions & Rationale

- Tuples for readings: a (day, temperature) pair is a natural immutable record -- once measured, a reading should not silently change, which matches the syllabus's emphasis on tuple immutability.
- Manual algorithm implementations: hottest/coldest-day finding and Kth-element selection are implemented with explicit loops/sorting rather than only using built-in `max()/min()`, to directly demonstrate the underlying algorithmic technique as taught.
- Ordered-array duplicate removal: duplicates are removed by first sorting, then comparing each element only to the previous one in a single pass -- the exact technique shown for 'Removal of Duplicates from an Ordered Array', rather than a set-based shortcut.
- No classes / no GUI: the entire project deliberately stays within the syllabus's actual scope (arrays, lists, tuples) rather than introducing OOP or external frameworks not yet covered.
- Plain text file storage: avoids the overhead of a database or JSON library for what is fundamentally a small, line-oriented dataset.

8. Implementation Details

The project is implemented in Python 3 across 7 files:

File	Responsibility
main.py	Menu-driven CLI entry point
array_algorithms.py	Reversal, counting, max/min, duplicate removal, partitioning, Kth smallest
tuple_operations.py	Tuple packing/unpacking, concatenation, slicing, membership, comparison
data_manager.py	List operations: append, insert, remove, traverse, slice
report.py	Text summary report and ASCII bar chart generation
storage.py	Save/load readings as a plain text file
validators.py	Centralized input validation and <code>ValidationError</code>

Core duplicate-removal algorithm (from `array_algorithms.py`), matching the syllabus's ordered-array technique:

```
ordered = sorted(readings, key=lambda r: r[0])
result = []
for reading in ordered:
    if not result or reading[0] != result[-1][0]:
        result.append(reading)
```

9. Screenshots / Results

Example run: analyzing 10 days of temperature readings, showing the generated summary report and ASCII bar chart.

```
===== WEATHER DATA ANALYSIS REPORT =====
Total readings: 10
Hottest day: Day 5 (40.0C)
Coldest day: Day 10 (-2.0C)
Average temperature: 21.7C

Category counts:
  Cold (<15C): 3
  Mild (15-30C): 4
  Hot (>30C): 3

Temperature chart:
Day  1 | ##### 18.0C
Day  2 | ##### 22.0C
Day  3 | ##### 35.0C
Day  4 | ##### 12.0C
Day  5 | ##### 40.0C
Day  6 | ##### 28.0C
Day  7 | ##### 9.0C
Day  8 | ##### 31.0C
Day  9 | ##### 24.0C
Day 10 | # -2.0C
```

Figure 6: Sample generated report output

10. Testing Approach

The core algorithms -- the heart of the application -- are covered by 14 unit tests (tests/test_array_algorithms.py) using Python's unittest framework, covering: array reversal, category counting, hottest/coldest detection, ordered duplicate removal, threshold partitioning, Kth coldest/hottest selection (including an out-of-range K raising a validation error), and all core tuple operations (packing, unpacking, concatenation, slicing, membership, comparison). All 14 tests pass, and the full CLI menu was also verified with a simulated end-to-end session covering every menu option.

Run with: `python -m unittest discover tests`

11. Challenges Faced

- Deciding how to apply 'Removal of Duplicates from an Ordered Array' (originally shown for numeric arrays) to tuple records -- solved by sorting on the day field first, then comparing adjacent day values only.
- Keeping the project strictly within the syllabus's scope (no classes, no external libraries) while still producing a usable, well-structured application -- solved with a purely functional, module-per-concern design.
- Designing a text-based visualization that still gives useful insight without a charting library -- solved with a proportional ASCII bar chart.

12. Learnings & Key Takeaways

- Classic array algorithms (reversal, counting, extremes, duplicate removal, partitioning, Kth-element selection) apply directly to real, everyday data once framed correctly.
- Tuples are a natural fit for representing immutable records like a single day's measurement.

- A purely functional, module-per-concern design can be just as maintainable as an object-oriented one for a project of this scope.

13. Future Enhancements

- Support for multiple named weather stations or datasets.
- Weekly/monthly trend summaries and moving averages.
- CSV export compatible with spreadsheet tools.
- An optional Tkinter graphical front-end once GUI concepts are covered.

14. References

- Course material: Array and its Operations, List and its Operations, Tuple and its Operations (Introduction to Problem Solving syllabus).
- Python Software Foundation. Python 3 Documentation - list, tuple, and file I/O. <https://docs.python.org/3/>